Department of Software Engineering, NED University of Engineering & Technology
SE-303 Operating Systems - Lab Session 13 (Complex Engineering Activity)

A Multi-Level Feedback Queue Scheduler for xv6-RISC-V and Its Effect on CPU-Bound and I/O-Bound Programs

Group of two
Muhammad Ashhadullah Zaheer (SE-24087)
Misbah Sarfraz (SE-24096)
Batch 2024-2028

1. Objective

The xv6 kernel schedules processes round-robin: every runnable process gets the CPU in turn, whatever it is doing. This lab replaces that policy with a Multi-Level Feedback Queue (MLFQ), adds a system call that exposes per-process CPU accounting, and uses two user programs (one that computes, one that does file I/O) to measure what the change does to each kind of workload. The report covers the design, how it was tested, the measurements, and what they say about the relationship between a kernel scheduling decision and the way applications should be written.

2. Environment and method

Item	Value
Operating system	xv6-riscv from github.com/mit-pdos/xv6-riscv, commit fa2ddcb
Emulator	QEMU 8.2.2, machine "virt", 128 MB RAM, virtio disk
Toolchain	riscv64-linux-gnu-gcc 13.3.0 on Ubuntu 24.04
Scheduler comparison runs	one hart (-smp 1) so that the two schedulers compete for the same single CPU
Clock	QEMU -icount shift=2: virtual time is derived from executed instructions, not from the host clock
Timer tick	1,000,000 cycles = 100 ms of virtual time (unchanged from stock xv6)
Repetitions	3 runs of every scenario and kernel variant

Three kernels were built from the same source tree: **RR** (the original round-robin loop, selected with `make SCHED=RR`), **MLFQ** (the final design), and **MLFQ-nopreempt** (MLFQ with one feature switched off, used only to see which part of the design matters). The accounting code is identical in all three, so any difference in the numbers comes from the scheduling decision.

The clock setting deserves a note. A first set of runs used `-icount shift=2,sleep=off`. In that mode QEMU jumps the virtual clock forward whenever the CPU is idle, so a process waiting for the disk appears to wait for many ticks. A solo I/O run took over 5,500 ticks instead of about 34, and the figures varied from run to run. Those results were discarded. With the default idle behaviour the repeated runs agree to within a tick or two, which is what makes the comparison meaningful.

Baseline. The unmodified xv6 source was built separately and booted to the shell (logs/baseline_unmodified_boot.txt). The round-robin kernel above is the same code with the accounting added, and its solo results (Section 6, Table 1) are the baseline numbers.

3. Design and implementation

3.1 The scheduler

There are three priority levels, 0 being the highest. Time slices are 1, 2 and 4 ticks for levels 0, 1 and 2, so the processes that are expected to be short get the most frequent turns and the ones that keep computing

switch less often. The rules are:

- A new process starts at level 0 (set in `allocproc()`, so a child of a demoted parent also starts at the top).
- The scheduler always runs a runnable process from the highest non-empty level. Inside a level, the process that ran least recently goes first, which gives round-robin among equals.
- A process that is preempted after using its whole slice moves down one level; level 2 is the floor.
- A process that gives up the CPU before its slice is over (it sleeps on disk, a pipe, or `pause()`, or exits) keeps its level and starts its next slice from zero.

Two additions go beyond the lab text. **Priority boost:** every 100 ticks all processes return to level 0, otherwise a process demoted during a long computation could be starved indefinitely by a stream of interactive jobs. **Preempt on wakeup:** when `wakeup()` makes a process runnable that has a better level than the process running on that CPU, the CPU is told to reschedule when its current trap finishes. xv6 only reschedules on the timer interrupt, so without this a woken I/O process waits up to 100 ms behind a running CPU-bound process however high its priority is. Sections 6 and 7 show how much this matters.

3.2 Accounting by measured run time

The first version charged one tick to whichever process was running when the timer fired and demoted it when its tick count reached the slice length. The functional test caught the problem: an I/O-bound process that ran for a few microseconds per wakeup was being demoted simply because the timer sometimes fired during one of those microseconds. The final version reads the RISC-V `time` register before and after every dispatch (in `scheduler()`) and adds the actual run length to the process. If the process comes back still runnable it was preempted and the time counts toward its slice; if it comes back sleeping or exiting, the slice counter is cleared. The same measurement feeds `getprocinfo()`, so CPU time is exact rather than sampled.

3.3 The getprocinfo system call

`int getprocinfo(int pid, struct procinfo *info)` fills a structure with the pid, process state, scheduler mode, current queue, CPU ticks and milliseconds, number of times scheduled, and CPU milliseconds spent at each level. A pid of zero or less means the caller. xv6 system calls return a single integer, so the lab's `getprocinfo(int pid)` had to gain an output pointer to return several metrics. The kernel copies the data out under the process lock and calls `copyout()` after releasing it. It returns -1 for an unknown pid or a bad user pointer.

3.4 Locking and kernel integration

All new fields live in `struct proc` and are protected by `p->lock`, as the rest of that structure is. The pick function scans the table once taking each lock briefly, then locks the winner and checks it is still runnable before switching to it; if another CPU got there first it scans again. The only unlocked access is the priority comparison in `wakeup()`, which sets a per-CPU hint flag. A stale read there can at worst cause one unnecessary or one delayed reschedule. The new system call follows the existing pattern (number in syscall.h, table entry in syscall.c, handler in sysproc.c, stub in usys.pl, prototype in user.h).

File	Change
kernel/proc.c, proc.h	MLFQ pick, run-time accounting, boost, wakeup preemption hint, kgetprocinfo(), new per-process fields
kernel/trap.c	trap exit also yields when a preemption was requested; boost called from the clock interrupt
kernel/sysproc.c, syscall.c, syscall.h, user/usys.pl, user/user.h	getprocinfo system call
kernel/procinfo.h, param.h, defs.h	shared result structure, tunables (NQUEUE, MLFQ_BOOST_TICKS, TICK_CYCLES), prototypes
Makefile	SCHED=RR and NOPREEMPT=1 build switches, new user programs

File	Change
user/cpubound.c, iobound.c, workloads.h	the two test applications and their shared workload code
user/bench.c, mlfqtest.c, mlfqdemo.c	benchmark runner, functional tests, scheduling demonstration

4. Test applications

cpubound counts the primes below 4,000,000 by trial division. It makes no system calls, so it never blocks and always uses its whole slice. On one hart it needs about 11.4 s of CPU (115 ticks).

iobound runs 100 iterations of: create a file, write four 64-byte blocks, close it, delete it. Each of those calls goes through the xv6 log and the virtio disk and puts the process to sleep, so the program is dispatched about 4423 times during one run and holds the CPU for a very short time on each. Solo it takes 33 ticks.

bench N M forks N copies of the first and M of the second and starts them together. Each child records its start and finish time and its `getprocinfo()` counters, and sends them to the parent through a pipe, so the console output of different processes never interleaves. An early version of the CPU workload finished in 0 ms because the compiler removed the loop whose result was unused; the result is now stored in a volatile variable.

5. Functional testing

`mlfqtest` runs 17 checks; all of them pass on the MLFQ kernel with one hart and with three harts (17 and 17 passes, no failures). They are listed below.

#	Check	Result
1	getprocinfo(self) succeeds	PASS
2	reported pid equals getpid()	PASS
3	running on MLFQ build	PASS
4	new process starts at top priority (0)	PASS
5	nsched >= 1 for a running process	PASS
6	getprocinfo(explicit own pid) matches	PASS
7	getprocinfo(bad pid 99999) returns -1	PASS
8	getprocinfo(bad pointer) returns -1	PASS
9	cpu_ticks grows while spinning	PASS
10	CPU hog demoted below level 0	PASS
11	CPU hog reaches the bottom queue	PASS
12	level_ms add up to cpu_ms	PASS
13	time spent in bottom queue is recorded	PASS
14	child of demoted parent starts at priority 0	PASS
15	I/O-bound process stays at priority 0	PASS
16	sleeping process stays at priority 0	PASS
17	periodic boost returns demoted process to level 0	PASS

For stability, the standard xv6 test suite (`usertests -q`) was run on the MLFQ kernel with three harts. It ends with "ALL TESTS PASSED" and the log contains no kernel panic (logs/usertests_mlfq.txt).

Scheduling demonstration

`mlfqdemo` starts two CPU-bound processes and one I/O-bound process and prints their level, CPU ticks and number of dispatches every two ticks. The excerpts below show the same program under both kernels (columns: elapsed ticks; for each process its pid, queue level, CPU ticks and dispatch count).

Round-robin kernel

```
2 | cpu-A 4  L0  0  1 | cpu-B 5  L0  1  1 | io    6  L0  0  1 |
4 | cpu-A 4  L0  1  2 | cpu-B 5  L0  2  2 | io    6  L0  0  2 |
6 | cpu-A 4  L0  2  3 | cpu-B 5  L0  3  3 | io    6  L0  0  3 |
8 | cpu-A 4  L0  3  4 | cpu-B 5  L0  4  4 | io    6  L0  0  4 |
10 | cpu-A 4  L0  4  5 | cpu-B 5  L0  5  5 | io    6  L0  0  5 |
12 | cpu-A 4  L0  5  6 | cpu-B 5  L0  6  6 | io    6  L0  0  6 |
14 | cpu-A 4  L0  5  7 | cpu-B 5  L0  7  7 | io    6  L0  0  7 |
16 | cpu-A 4  L0  6  8 | cpu-B 5  L0  8  8 | io    6  L0  0  8 |
18 | cpu-A 4  L0  7  9 | cpu-B 5  L0  9  9 | io    6  L0  0  9 |
```

MLFQ kernel

```
2 | cpu-A 4  L0  0  1 | cpu-B 5  L0  0  1 | io    6  L0  0  1 |
4 | cpu-A 4  L1  1  2 | cpu-B 5  L1  1  2 | io    6  L0  0  2 |
6 | cpu-A 4  L1  1  124 | cpu-B 5  L1  2  123 | io    6  L0  1  246 |
8 | cpu-A 4  L1  1  243 | cpu-B 5  L1  2  243 | io    6  L0  2  487 |
10 | cpu-A 4  L1  1  357 | cpu-B 5  L1  2  357 | io    6  L0  3  717 |
12 | cpu-A 4  L1  2  474 | cpu-B 5  L1  2  473 | io    6  L0  5  952 |
14 | cpu-A 4  L1  2  589 | cpu-B 5  L1  3  588 | io    6  L0  6  1184 |
16 | cpu-A 4  L1  2  705 | cpu-B 5  L1  3  705 | io    6  L0  7  1419 |
18 | cpu-A 4  L1  2  818 | cpu-B 5  L1  3  818 | io    6  L0  8  1646 |
```

Under round-robin every process stays at level 0 and the I/O process has been dispatched only 9 times by tick 18: after each disk operation it queues behind the two CPU-bound processes. Under MLFQ the CPU-bound processes are pushed down the queues while the I/O process stays at level 0 and has been dispatched 1646 times by tick 18.

6. Benchmark results

Times are completion times in timer ticks (1 tick = 100 ms of virtual time) measured by each process from its own start to its own end, as the mean of 3 runs with the range in brackets when the runs differ. The CPU time column is the process's own CPU consumption from getprocinfo(); dispatches is how often the scheduler switched to it.

Table 1. Each program alone (baseline)

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, round-robin	115	11437	116	0
CPU-bound, MLFQ	115	11436	116	2
I/O-bound, round-robin	33 (33-34)	2380	4423	0
I/O-bound, MLFQ	35	2381	4426	0

Table 2. One CPU-bound and one I/O-bound process started together

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, round-robin	116	11470	117	0
I/O-bound, round-robin	147 (147-148)	2380	4426	0
CPU-bound, MLFQ	145 (145-146)	11638	4511	2
I/O-bound, MLFQ	37 (37-38)	2381	4426	0
CPU-bound, MLFQ without wakeup preemption	116	11464	117	2
I/O-bound, MLFQ without wakeup preemption	149	2380	4424	0

Table 3. Three CPU-bound and one I/O-bound process started together

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, round-robin	343 (342-345)	11433	116	0
I/O-bound, round-robin	375 (374-375)	2380	4423	0
CPU-bound, MLFQ	374 (372-375)	11506	1581	2
I/O-bound, MLFQ	39	2381	4426	0

Process / kernel	Completion (ticks)	CPU time (ms)	Dispatches	Final level
CPU-bound, MLFQ without wakeup preemption	346 (345-347)	11457	117	2
I/O-bound, MLFQ without wakeup preemption	377 (376-377)	2380	4425	0

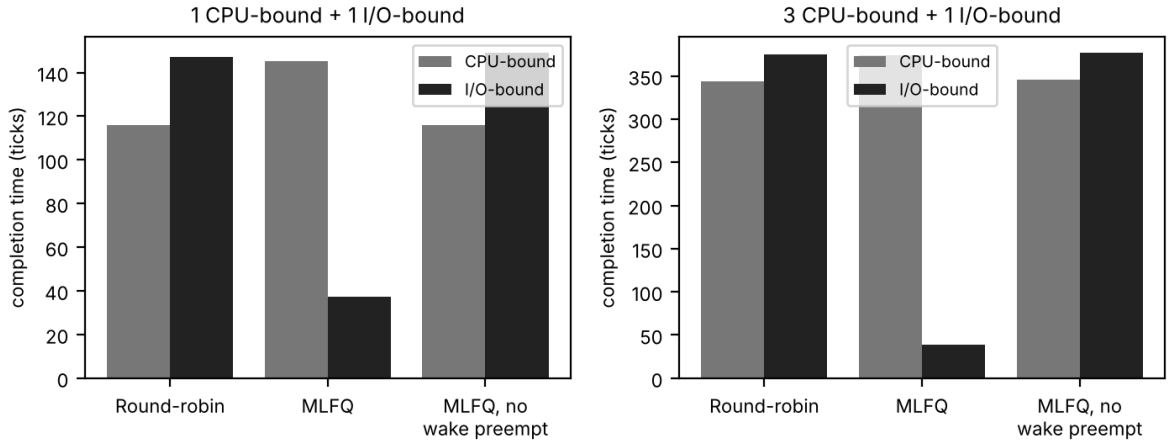


Figure 1. Completion time of the two kinds of process under each kernel (mean of 3 runs).

Table 4. Summary of the two mixed runs

Measure	1 CPU + 1 I/O: RR	1 CPU + 1 I/O: MLFQ	3 CPU + 1 I/O: RR	3 CPU + 1 I/O: MLFQ
I/O-bound completion (ticks)	147	37	375	39
I/O-bound slowdown versus running alone	4.4x	1.1x	11.2x	1.2x
CPU-bound completion (ticks, mean)	116	145	343	374
Mean completion of all processes (ticks)	132	91	351	290
Time until the last process ends (ticks)	147	145	375	375

7. Analysis: why the two kinds of process are affected differently

The I/O-bound program gains a great deal. Alone it finishes in 33 ticks. Next to three CPU-bound processes under round-robin it needs 375, about 11x longer, and it only finishes after the CPU-bound processes are done. It needs about 4423 dispatches, but every time it blocks and wakes it has to wait for its turn behind the other runnable processes, and each turn starts at a timer tick. Under MLFQ the same program finishes in 39 ticks, 1.2x its solo time. It never uses a whole slice, so it never leaves level 0 (its final level in every run is 0), and the scheduler picks it ahead of anything at lower levels.

The CPU-bound programs pay a small price. Each one burns its slice every time, so each is pushed down to level 2 (final level 2 in every MLFQ run) and runs only when nothing above it is runnable. In the three-process case their completion time goes from 343 to 374 ticks (+8.8%). The I/O process only needs about 2.4 s of CPU in total, so the extra delay is roughly that amount plus context-switch overhead: the CPU-bound processes are dispatched 1581 times each instead of 116, because every I/O wakeup preempts them. The last process finishes at almost the same time under both kernels (375 against 375 ticks), so MLFQ does not waste CPU; it changes the order in which the work is done.

Mean completion time improves. Finishing the short jobs first lowers the average: from 351 to 290 ticks with three CPU-bound processes, and from 132 to 91 with one. This is the same effect as shortest-job-first, obtained without knowing job lengths in advance: the scheduler infers from past behaviour that a process which keeps blocking is short-burst work.

Preempting on wakeup is what makes the I/O result possible. With the feature removed, MLFQ gives 377 ticks for the I/O process in the three-process case and 149 in the two-process case, the same as round-robin (375 and 147). Priorities by themselves only matter at the moments the scheduler runs, and stock xv6 runs it once per tick. A priority scheme on top of a tick-driven kernel therefore needs a way to run the scheduler when a higher-priority process becomes runnable; the kernel-side mechanism is what determines whether the policy has any effect on applications.

Round-robin versus MLFQ at the process level. Round-robin treats the two programs identically, so the one that spends most of its life waiting is penalised most: its waits get longer by a factor of the number of competitors. MLFQ separates them by observed behaviour, with no information from the programs themselves.

8. Conclusion: what this means for application design

- **Blocking early is rewarded.** A program that spends its time waiting, and computes in short bursts, keeps a high priority and responds quickly even on a busy machine. A program that is a long computation should expect to run at low priority when anything interactive is present. In practice that favours structures such as an event loop that waits for input and does small pieces of work, and it makes a batch job that tries to be responsive by polling in a busy loop work against itself.
- **Long computations should expect to be preempted often.** The CPU-bound processes were dispatched about 1581 times instead of 116. Work that keeps state in memory and can restart cheaply after being switched out suffers little; work that depends on long uninterrupted cache-friendly runs loses more.
- **The policy can be gamed.** A CPU-bound program that sleeps or does a trivial I/O call just before its slice ends is treated as interactive and stays at level 0. The periodic boost does not prevent that; it only prevents starvation. Real systems limit this by charging total time at a level rather than per slice, a rule not implemented here.
- **Mixed programs are classified by their recent past.** A process that computes for a long time and then starts doing I/O stays at a low level until the next boost, so its first responses after the change are slow. A process that does I/O first and computes later is demoted quickly. Splitting the I/O handling and the computation into separate processes lets each be scheduled for what it does.
- **Parameters are part of the design.** The slice lengths (1, 2, 4 ticks) and the boost period (100 ticks) were chosen by reasoning and were not tuned; a different tick length or I/O latency could change the numbers above, and the benchmark setup makes them easy to vary.

9. Limitations

Results are from an emulated machine with a fast virtual disk, one hart, and instruction-count-based virtual time; absolute tick values on real hardware or another QEMU version will differ, and the ratios are the meaningful part. Only two workload types were used. The scheduler scans all 64 process slots on every decision, as the original loop did, so it is not designed for large process tables. The stress test (usertests) was run once with three harts, and the benchmarks with one hart; multi-hart benchmark behaviour was checked only through the functional test and usertests.

10. Reproducing the results

```
cd xv6-mlfq-lab
sudo apt install qemu-system-misc gcc-riscv64-linux-gnu make python3 python3-reportlab python3-matplotlib
bash scripts/build_all.sh                  # builds build/kernel-RR, -MLFQ, -MLFQ-nopreempt
python3 scripts/run_benchmarks.py 3        # writes logs/raw/*.txt and results/results.csv
python3 scripts/collect_logs.py            # functional test, demo, usertests logs
python3 scripts/make_report.py             # rebuilds this PDF from the data

# interactive use:  cd xv6-riscv && make qemu CPUS=1   (MLFQ)
#                   make clean && make qemu SCHED=RR CPUS=1   (round-robin)
```

NED University of Engineering & Technology
Department of Software Engineering
CEA RUBRICS
(SE-303) Operating Systems

Student: Muhammad Ashhadullah Zaheer (SE-24087)

Criteria	Good (2)	Satisfactory (1)	Not Achieved (0)	CPA	Marks
1. Scheduler Logic & Correctness	The MLFQ scheduler is implemented correctly and efficiently, adhering to all specified logic (priority levels, promotion/demotion rules).	The scheduler is functional but contains minor logical flaws or inefficiencies. It may not fully handle all edge cases correctly.	The scheduler fails to compile, is non-functional, or does not implement the core requirements of an MLFQ.	CPA1 CPA2 CPA3	
2. Kernel Integration & Stability	The new scheduler and system call are cleanly integrated into the kernel without causing instability, memory leaks, or race conditions.	The new code is functional but may cause minor, intermittent issues or does not follow xv6's existing design patterns.	The implementation causes frequent kernel panics, data corruption, or fails to integrate with the existing kernel code.	CPA1 CPA2 CPA3	
3. Test Application Design	Both the CPU-bound and I/O-bound applications are well-designed, effectively simulating their respective workloads to clearly test the scheduler.	The applications are functional but are not distinct enough in their behavior, making the performance difference less clear.	The applications fail to run, do not correctly simulate the intended workloads, or were not developed.	CPA1 CPA2 CPA3	
4. Functional Demonstration	The functionality of the new scheduler is clearly and correctly demonstrated, showing how it prioritizes processes differently than the original scheduler.	A demonstration is provided, but it is unclear or fails to convincingly show the impact of the new scheduling algorithm.	No functional demonstration is provided, or the demonstration shows the implementation does not work.	CPA1 CPA3	
5. Code Quality & Readability	The code written (both kernel and user-level) is clean, well-commented, easy to understand, and follows standard C programming practices.	The code is functional but is poorly formatted, lacks sufficient comments, or is difficult to follow.	The code is unreadable, uncommented, and does not follow basic programming standards.	CPA3	
Weighted CLO Score					
Remarks and Signature					

NED University of Engineering & Technology
Department of Software Engineering
CEA RUBRICS
(SE-303) Operating Systems

Student: Misbah Sarfraz (SE-24096)

Criteria	Good (2)	Satisfactory (1)	Not Achieved (0)	CPA	Marks
1. Scheduler Logic & Correctness	The MLFQ scheduler is implemented correctly and efficiently, adhering to all specified logic (priority levels, promotion/demotion rules).	The scheduler is functional but contains minor logical flaws or inefficiencies. It may not fully handle all edge cases correctly.	The scheduler fails to compile, is non-functional, or does not implement the core requirements of an MLFQ.	CPA1 CPA2 CPA3	
2. Kernel Integration & Stability	The new scheduler and system call are cleanly integrated into the kernel without causing instability, memory leaks, or race conditions.	The new code is functional but may cause minor, intermittent issues or does not follow xv6's existing design patterns.	The implementation causes frequent kernel panics, data corruption, or fails to integrate with the existing kernel code.	CPA1 CPA2 CPA3	
3. Test Application Design	Both the CPU-bound and I/O-bound applications are well-designed, effectively simulating their respective workloads to clearly test the scheduler.	The applications are functional but are not distinct enough in their behavior, making the performance difference less clear.	The applications fail to run, do not correctly simulate the intended workloads, or were not developed.	CPA1 CPA2 CPA3	
4. Functional Demonstration	The functionality of the new scheduler is clearly and correctly demonstrated, showing how it prioritizes processes differently than the original scheduler.	A demonstration is provided, but it is unclear or fails to convincingly show the impact of the new scheduling algorithm.	No functional demonstration is provided, or the demonstration shows the implementation does not work.	CPA1 CPA3	
5. Code Quality & Readability	The code written (both kernel and user-level) is clean, well-commented, easy to understand, and follows standard C programming practices.	The code is functional but is poorly formatted, lacks sufficient comments, or is difficult to follow.	The code is unreadable, uncommented, and does not follow basic programming standards.	CPA3	
Weighted CLO Score					
Remarks and Signature					